

ANALYSE PERSONNELLE DE LA DYNAMIQUE DU PROJET

*BC2 — Coordonner une équipe projet
Mobilisation d'une équipe projet et conduite du changement
Épreuve individuelle — 20% de la note BC2*

Nom et Prénom	Elyes JAFFEL
Promotion	CPI 2025 – 2026 · Full Stack Dev
Groupe projet	Groupe Omar Akakba / Elyes Jaffel
Spécialisation	DEV
Date de rendu	21/06/2026

Document individuel — 8 à 10 pages de texte — rendu après la présentation orale collective (épreuve 2.1)

TABLE DES MATIÈRES

1. Mon rôle dans le projet CYNA
 - 1.1 Poste, responsabilités et périmètre
 - 1.2 Organisation au quotidien avec le groupe
2. Ma contribution technique et mes apprentissages
 - 2.1 Livrables et productions personnels
 - 2.2 Valeur ajoutée spécifique
 - 2.3 Ce que j'ai appris des autres membres
3. Dynamique collective : ce qui a fonctionné

- 3.1 Facteurs d'efficacité collective
 - 3.2 Prise en compte des différences entre membres
 - 3.3 Gestion des désaccords
 - 4. Ce qui a été difficile
 - 4.1 Principale difficulté technique
 - 4.2 Principale difficulté organisationnelle ou relationnelle
 - 4.3 Les moments où j'ai été un frein
 - 4.4 Ce que ces difficultés ont changé
 - 5. Bilan critique et enseignements
 - 5.1 Vision du projet abouti
 - 5.2 Ce que j'aurais fait différemment
 - 5.3 Enseignements durables
 - 5.4 Projection professionnelle
-

1. MON RÔLE DANS LE PROJET CYNA

1.1 Poste, responsabilités et périmètre

J'ai occupé le rôle de Lead Développeur Front-End sur le projet CYNA, une plateforme SaaS de cybersécurité développée en PHP 8.2 natif, permettant à des entreprises d'acheter des solutions EDR, SOC Managé et VPN en abonnement mensuel ou annuel.

Mon périmètre couvrait l'ensemble de la couche interface utilisateur : conception de la charte graphique CYNA sur Figma (palette de couleurs, typographie, composants UI réutilisables), production des wireframes et maquettes haute fidélité pour desktop et mobile, intégration HTML5/CSS3 de toutes les pages avec Bootstrap 5, création du fichier `assets/css/style.css` regroupant le style personnalisé de la plateforme, et développement de l'interactivité JavaScript dans `assets/js/main.js` (burger menu mobile, auto-hide des alertes, gestion du panier, filtres du catalogue, validation des formulaires côté client). J'étais aussi responsable du rendu responsive et de la compatibilité multi-navigateurs.

Sur les tâches transverses, Omar et moi nous sommes entraides : j'ai participé à la définition de la structure des pages de l'espace client et du backoffice admin, qui

nécessitaient une coordination étroite entre la logique PHP backend et le rendu visuel. Omar m'a de son côté aidé à comprendre les contraintes des fichiers PHP natifs — notamment le fonctionnement des sessions et du panier stocké dans `$_SESSION` — pour que mon JavaScript s'y adapte correctement.

1.2 Organisation au quotidien avec le groupe

Nous étions deux sur ce projet — Omar au back-end et moi au front-end — ce qui impliquait une communication directe et fréquente, sans intermédiaire. Nos échanges quotidiens se faisaient via Discord, avec des points réguliers pour valider les avancées et identifier les blocages. GitHub était notre outil central : chaque fonctionnalité faisait l'objet d'une branche dédiée, les commits étaient préfixés (`feat:` , `fix:` , `docs:`), et les Pull Requests permettaient une revue croisée avant chaque merge.

La répartition était claire dès le départ — Omar possédait le backend PHP et la base de données MySQL, je possédais le HTML, le CSS et le JavaScript. La structure PHP natif du projet — des fichiers directs comme `catalogue.php` , `produit.php` , `panier.php` intégrant chacun `includes/header.php` et `includes/footer.php` — nous permettait de travailler en parallèle : Omar gérait la logique PHP en haut de chaque fichier, et j'intégrais le HTML/CSS en dessous. Cette séparation claire dans le même fichier a bien fonctionné dans la majorité des cas, même si elle a parfois généré des conflits de merge que nous avons résolus ensemble.

2. MA CONTRIBUTION TECHNIQUE ET MES APPRENTISSAGES

2.1 Livrables et productions personnels

Ma première contribution a été la conception de l'identité visuelle CYNA sur Figma : palette de couleurs sombre à dominante bleu marine (`#060c1b` , `#0d2044`) cohérente avec le positionnement cybersécurité, typographie, et composants UI réutilisables (boutons, cartes produit, badges de disponibilité, formulaires, modals). Ces maquettes haute fidélité pour desktop et mobile ont servi de référence tout au long du développement pour garantir la cohérence visuelle entre chaque page.

J'ai ensuite intégré l'ensemble des pages HTML5/CSS3 en PHP natif avec Bootstrap 5 : page d'accueil avec carrousel JavaScript personnalisé, catalogue avec filtres par catégorie

et pagination, fiche produit, tunnel de commande (panier → checkout → confirmation), espace client (historique commandes, gestion des adresses, modification du profil), et panel d'administration (tableau de bord, gestion des produits, commandes, messages de contact, carousel). Le fichier `assets/css/style.css` centralise tous les styles personnalisés de la plateforme, complétant Bootstrap 5 avec la charte graphique CYNA.

En collaboration avec Omar sur les phases transverses, j'ai développé `assets/js/main.js` qui regroupe toutes les interactions JavaScript de la plateforme : burger menu mobile, auto-hide des alertes flash PHP, gestion dynamique du compteur de panier dans la navigation, et filtres du catalogue. J'ai également participé à la recette fonctionnelle en testant manuellement tous les parcours utilisateurs critiques (inscription, connexion, achat complet, suppression de compte RGPD) pour identifier les anomalies avant le rendu.

2.2 Valeur ajoutée spécifique

Ma valeur ajoutée principale a été la création d'une identité visuelle cohérente et professionnelle qui a donné à la plateforme CYNA une crédibilité en accord avec son positionnement sur la cybersécurité. Sans la charte graphique Figma définie en amont, le développement des pages aurait avancé sans ligne directrice commune et aurait produit une incohérence visuelle entre les différentes parties de l'application. Le fait d'avoir des maquettes validées avant le début de l'intégration a permis à Omar de se concentrer entièrement sur la logique PHP et MySQL sans se préoccuper du rendu visuel.

J'ai également apporté la couverture responsive et multi-navigateurs que le projet n'aurait pas eue sans moi : chaque page a été testée sur Chrome, Firefox et Edge, et la grille Bootstrap 5 a été configurée pour garantir un affichage correct sur mobile, tablette et desktop. Sur une plateforme SaaS où des clients professionnels accèdent depuis des équipements variés, cette couverture multi-device était un critère qualité directement lié à la crédibilité commerciale du produit.

2.3 Ce que j'ai appris des autres membres

Omar m'a transmis une culture de la sécurité que je n'appliquais pas spontanément côté front-end. En observant son approche — protection CSRF sur chaque formulaire via un token vérifié côté PHP, validation systématique des données côté serveur même quand la validation JavaScript côté client était déjà en place — j'ai compris que la sécurité ne se

divise pas entre front et back. Un formulaire avec une bonne validation JavaScript reste une faille si le PHP côté serveur ne revalide pas les données reçues. Cette prise de conscience a changé ma façon d'écrire mes formulaires HTML : je les traite désormais comme un premier niveau d'expérience utilisateur, pas comme une barrière de sécurité.

Sur la rigueur technique, Omar m'a montré l'importance de la traçabilité dans le travail quotidien : ses commits explicites avec préfixes conventionnels, ses fichiers PHP bien structurés avec des commentaires sur les choix de sécurité, sa gestion des variables sensibles dans `.env` — autant d'habitudes qui lui ont permis de déboguer rapidement et de me donner une visibilité claire sur l'état du backend. J'ai adopté cette discipline dans mon propre travail en structurant mon CSS par composant et en nommant mes fonctions JavaScript de manière descriptive, ce qui a simplifié les phases de relecture croisée entre nous.

3. DYNAMIQUE COLLECTIVE : CE QUI A FONCTIONNÉ

3.1 Facteurs d'efficacité collective

Ce qui a le mieux fonctionné dans notre binôme est la séparation naturelle qu'offrait la structure PHP natif du projet : dans chaque fichier comme `catalogue.php` ou `espace-client.php`, Omar gérait la logique PHP en haut du fichier (requêtes SQL, traitement des données, vérification de session) et je prenais la main sur le HTML/CSS en dessous. Cette organisation en deux zones dans le même fichier nous a permis d'avancer en parallèle sur la plupart des pages sans nous bloquer mutuellement.

Un autre facteur d'efficacité a été la décision de centraliser les éléments partagés dès le début : `includes/header.php` pour la navigation, le compteur de panier et les en-têtes de sécurité HTTP, `includes/footer.php` pour le pied de page et les scripts. Cette centralisation signifiait qu'une modification de la navigation ou du style global n'avait besoin d'être faite qu'en un seul endroit, ce qui a réduit considérablement le risque d'incohérence entre les pages.

Nous avons aussi utilisé GitHub non seulement comme outil de versioning mais comme espace de communication technique. Quand j'avais besoin qu'Omar crée ou modifie une requête SQL pour une page que j'intégrais, je le signalais dans le message de commit ou

via Discord avec le nom du fichier concerné. Cette traçabilité a permis d'éviter les malentendus sur ce qui était prêt à intégrer et ce qui ne l'était pas encore.

3.2 Prise en compte des différences entre membres

Nos profils étaient techniquement complémentaires mais culturellement différents : Omar avait une approche orientée robustesse et sécurité — il anticipait les cas d'erreur et les failles avant même que la fonctionnalité ne soit terminée. Mon approche était plus orientée expérience utilisateur — je pensais d'abord au parcours visuel et à la clarté de l'interface. Ces deux manières de travailler auraient pu créer des frictions, mais nous avons transformé cette différence en avantage : Omar m'alertait quand une interaction JavaScript que je proposais créait un risque de sécurité ou ignorait un cas d'erreur côté PHP, et je lui signalais quand une contrainte backend produisait un message d'erreur trop technique pour l'utilisateur final.

En pratique, cela s'est traduit par des sessions de relecture croisée où chacun sortait de son domaine pour regarder le résultat de l'autre. Ces moments ont produit des améliorations concrètes des deux côtés — des messages d'erreur plus clairs dans mes formulaires HTML, des réponses PHP plus adaptées à l'affichage dans mes pages — sans jamais empiéter sur la responsabilité de l'autre.

3.3 Gestion des désaccords

Un désaccord concret a porté sur le choix du système de paiement : je préférais PayPal pour sa simplicité d'intégration et sa reconnaissance immédiate par les utilisateurs, tandis qu'Omar défendait Stripe pour sa robustesse technique et sa gestion native des webhooks. De mon côté, l'argument principal était que PayPal réduirait le temps de développement et permettrait de consacrer plus de temps à l'interface du tunnel de paiement.

Omar a tranché l'argument en me montrant concrètement comment les webhooks Stripe permettent de mettre à jour automatiquement le statut de commande en base de données lorsque le paiement est confirmé côté Stripe — un mécanisme que PayPal gère moins proprement. Confronté à cet argument technique factuel, j'ai reconnu que la fiabilité du traitement backend était plus importante pour une plateforme SaaS que la familiarité de l'interface de paiement. Nous avons retenu Stripe. Ce désaccord m'a appris à peser les arguments techniques même quand ils vont à l'encontre de ma première intuition.

4. CE QUI A ÉTÉ DIFFICILE

4.1 Principale difficulté technique

La principale difficulté technique a été la synchronisation entre la gestion du panier en JavaScript côté client et la session PHP côté serveur. J'avais initialement conçu le compteur de panier en JavaScript pur en lisant le DOM pour compter les articles affichés, ce qui donnait un affichage réactif. Mais cette approche créait une incohérence : si la page se rechargeit ou que l'utilisateur naviguait vers une autre page, le compteur ne reflétait plus le vrai contenu du panier stocké dans `$_SESSION['cart']` côté PHP. Omar m'a expliqué que le panier réel vivait uniquement en session PHP, et que mon JavaScript devait afficher ce que le PHP lui communiquait, pas recalculer de son côté.

Il a fallu revoir l'architecture du compteur de panier dans `includes/header.php` : Omar a ajouté le calcul PHP du nombre d'articles en session (`$cart_count`) directement dans le header, et j'ai adapté mon JavaScript pour mettre à jour ce compteur visuellement sans recalculer la logique. Cette coordination a demandé plusieurs allers-retours entre nous, mais le résultat final était cohérent et fiable. J'ai retenu que dans une architecture PHP natif sans API REST, le JavaScript est une couche d'affichage et d'interaction — pas une source de vérité sur les données.

L'impact sur le groupe a été limité grâce à notre communication directe, mais le temps de débogage a décalé l'intégration d'autres pages. Si nous avions formalisé dès le début comment le PHP et le JavaScript allaient se partager la gestion des données, cette friction aurait été évitée.

4.2 Principale difficulté organisationnelle ou relationnelle

La principale difficulté organisationnelle a été la gestion des dépendances entre mes livrables visuels et les livrables PHP d'Omar. Plusieurs pages que j'intégrais en HTML/CSS nécessitaient que les requêtes SQL et les variables PHP correspondantes soient déjà en place pour afficher des données dynamiques. Il m'est arrivé d'intégrer des pages avec des données statiques en attendant le PHP, puis de devoir reprendre tout le HTML quand les variables PHP étaient disponibles mais structurées différemment de ce que j'avais anticipé.

Cette situation a mis en évidence un manque de coordination préalable sur la structure des données échangées entre nos deux parties. Nous aurions dû définir ensemble, pour chaque page, quelles variables PHP seraient disponibles et sous quelle forme, avant que je commence l'intégration HTML. À la place, nous communiquons ces informations au moment du développement, ce qui créait des reprises de travail évitables.

4.3 Les moments où j'ai été un frein

J'ai été un frein en premier lieu par des modifications de maquettes Figma intervenues trop tard dans le cycle de développement. À deux reprises, j'ai revu des choix de design — la structure du tableau de bord admin et la mise en page de la fiche produit — alors qu'Omar avait déjà structuré sa logique PHP en fonction de la version précédente. Ces changements visuels en apparence mineurs ont eu des conséquences sur l'organisation des données que PHP devait transmettre à la page, obligeant Omar à des ajustements non planifiés. J'avais sous-estimé à quel point une modification de la disposition des éléments HTML pouvait impacter la logique backend.

J'ai également été un frein par une mauvaise estimation du temps sur l'intégration JavaScript du panier. J'avais annoncé cette fonctionnalité réalisable en deux jours ; elle en a finalement pris cinq, compte tenu des aller-retours nécessaires avec Omar pour aligner le comportement JavaScript sur la logique de session PHP. Ce décalage a impacté le planning global et mis Omar en attente pour finaliser les tests du tunnel de commande.

Plus généralement, j'avais tendance à présenter mes maquettes comme "en cours d'amélioration" même après le début du développement, sans indiquer clairement ce qui était figé et ce qui pouvait encore changer. Ce manque de clarté sur le statut de mes livrables a créé des incertitudes pour Omar qui ne savait pas toujours sur quelle version de la page s'appuyer.

4.4 Ce que ces difficultés ont changé

Ces difficultés m'ont convaincu qu'un livrable de design ne doit jamais rester en statut "en cours" une fois que le développement backend a démarré dessus. Dans un prochain projet, je distinguerai formellement deux états pour mes maquettes : "draft — ne pas développer" et "validée — figée". Ce signal simple évite que l'autre membre investisse du temps sur une base qui va changer.

Sur la coordination technique, j'adopterai dans tout prochain projet un document de contrat front/back : pour chaque page, la liste des variables PHP disponibles et leur structure, définie ensemble avant le début de l'intégration HTML. Dans un projet PHP natif, ce contrat est particulièrement important car il n'y a pas de format d'API formalisé — la communication entre PHP et HTML se fait directement dans le même fichier, et une hypothèse incorrecte sur la structure d'une variable entraîne une reprise complète de l'intégration.

5. BILAN CRITIQUE ET ENSEIGNEMENTS

5.1 Vision du projet abouti

Ce qui me satisfait le plus dans le projet CYNA est la cohérence visuelle de la plateforme sur l'ensemble des pages : la charte graphique définie en début de projet sur Figma a tenu du premier au dernier livrable, et l'interface reflète effectivement le positionnement cybersécurité de CYNA — sobre, professionnelle, avec une palette sombre qui inspire confiance. Le fait que les pages soient responsives et testées sur plusieurs navigateurs représente également un niveau de finition que nous n'aurions peut-être pas atteint sans la discipline de recette maintenue jusqu'au bout.

Ce qui me déçoit est l'absence de tests automatisés côté front-end. J'avais prévu de tester les parcours critiques de manière systématique, mais la pression des délais en fin de projet m'a contraint à m'en tenir aux tests manuels. Ces tests couvrent les cas principaux mais ne détectent pas les régressions : une modification du PHP côté backend pourrait casser un affichage côté HTML sans que personne ne s'en aperçoive immédiatement.

Je suis également déçu par l'expérience du panier sur mobile : si la grille Bootstrap garantit un affichage correct, l'interaction d'ajout au panier sur petit écran est moins fluide qu'elle ne devrait l'être pour une plateforme SaaS commerciale. C'est un point que j'avais identifié lors des tests mais que je n'ai pas eu le temps de corriger avant le rendu.

5.2 Ce que j'aurais fait différemment

Sur le plan individuel, j'aurais gelé les maquettes Figma dès la validation du premier sprint et refusé toute modification structurelle après le début de l'intégration HTML. Le

coût d'une modification de design en phase de développement est bien plus élevé qu'en phase de maquettage — c'est une réalité que je connaissais en théorie mais que j'ai sous-estimée en pratique. Je consacrerai davantage de temps à la phase de maquettage dans un prochain projet pour livrer des interfaces plus robustes dès le départ.

Sur le plan collectif, j'aurais proposé dès le début une réunion de cadrage technique front/back pour chaque fonctionnalité majeure, afin de définir ensemble la structure des données PHP avant de commencer l'intégration HTML. Dans un projet PHP natif où le backend et le frontend cohabitent dans les mêmes fichiers, cette coordination préalable est encore plus critique que dans une architecture avec une API séparée. Nous avions les outils de communication (Discord, GitHub) mais nous n'avions pas instauré ce réflexe de cadrage systématique.

5.3 Enseignements durables

Sur le plan technique, j'ai appris à concevoir l'interface en tenant compte des contraintes backend dès la phase de maquettage. Avant CYNA, je pensais le design indépendamment de la logique serveur et j'ajustais ensuite lors de l'intégration. Après CYNA, je pense systématiquement à la source des données que j'affiche, à ce que PHP peut ou ne peut pas me fournir, avant de finaliser un composant HTML. J'ai également approfondi la maîtrise de Bootstrap 5 en contexte de projet réel — grille responsive, composants navbar, cards, modals — et compris comment intégrer un CSS personnalisé en complément sans créer de conflits.

Sur le plan humain, l'enseignement le plus durable est la valeur de la communication proactive sur le statut de ses livrables. Dans un binôme où chacun dépend de l'autre, ne pas signaler à temps qu'un livrable est en révision pénalise l'autre membre autant que soi-même. Cette leçon de transparence — communiquer ce qu'on fait, même quand c'est "je refais" — est directement transposable dans n'importe quelle configuration d'équipe en entreprise.

5.4 Projection professionnelle

Les compétences acquises sur CYNA que je vais mobiliser directement en entreprise sont, en priorité : la conception d'une charte graphique complète sur Figma avec composants réutilisables, l'intégration HTML5/CSS3 responsive avec Bootstrap 5 dans un environnement PHP natif, et le développement JavaScript d'interfaces interactives synchronisées avec un backend. Ces savoir-faire constituent le cœur du profil Front-End

développeur attendu en alternance ou en premier emploi, et je peux les démontrer avec le dépôt GitHub du projet CYNA comme preuve tangible.

Sur le plan comportemental, j'adopterai deux pratiques systématiques dans mon prochain projet : geler formellement mes livrables de design avant de les transmettre au backend, et définir avec le développeur backend la structure des données pour chaque page avant de commencer l'intégration HTML. Ces deux pratiques, que j'ai apprises à mes dépens sur CYNA, transforment une collaboration informelle qui fonctionne bien en binôme en une organisation qui peut s'adapter à des équipes plus grandes. C'est cette capacité à structurer la collaboration — et pas seulement à produire du code — que je veux continuer à développer pour évoluer vers un rôle de lead front-end.

Analyse Personnelle de la Dynamique du Projet — BC2 — CPI Bachelor / INGETIS — Elyes JAFFEL